Django template tags, with the book catalog

A reference and teaching guide: every commonly used tag and filter, each shown on the Book, Author, Genre and Language models of the catalog.

0. What the templates receive

All examples assume these names reach the template:

Name	Comes from	What it is
`books / page_obj`	List view	Paginated Book queryset. page_obj iterates like a list and carries paging info.
`book`	Detail view	One Book. Related data: `book.authors.all`, `book.genres.all`, `book.language`.
`q`	List view	The search text from the query string.

Fields used: Book (title, isbn, summary, publication_year, pages, cover, available, created_at), Author (first_name, last_name, full_name, birth_date, death_date, bio), Genre (name), Language (name, code).

1. The three syntaxes and the lookup rule

```
{{ book.title }}          print a value (variable)
{% if book.available %}   logic, loops, includes (tag)
{# short note #}          comment, never sent to the browser
```

For `book.language.name` Django tries, at every dot: dictionary lookup, then attribute, then list index. If the result is callable it is called **without arguments**. That is why you write `book.authors.all` without parentheses, and why a method that needs an argument cannot be called from a template.

Failure is silent. A misspelled variable prints nothing and raises no error. Also, `{{ book.authors }}` prints `catalog.Author.None`: that is the manager, not the authors. Use `.all`.

2. Structure tags

Tag	What it does	Catalog example
`extends`	Makes this template a child of another. Must be the first tag in the file. Anything outside a block is ignored.	`{% extends "base.html" %}`
`block / endblock`	Defines a replaceable region. The child fills it, the parent shows it.	`{% block content %}...{% endblock %}`
`block.super`	Inside a child block, prints what the parent had in that block.	`{% block title %}Books - {{ block.super }}{% endblock %}`
`include`	Renders another template inside this one, with the current context. `with` adds variables, `only` hides everything else, so dependencies are explicit.	`{% include "catalog/_book_card.html" with book=book only %}`
`load`	Enables a tag library. It is **not inherited**: load it in every template that uses it.	`{% load static %}` `{% load humanize %}`
`comment`	Multi-line comment, the content is skipped completely.	`{% comment "TODO" %}...{% endcomment %}`

3. Logic tags

Tag	What it does	Catalog example
`if / elif / else`	Conditions with `== != < > <= >= in not in and or not`. No parentheses allowed. A comparison that fails (for example `None > 500`) is simply false.	`{% if book.available %}...{% elif book.pages > 500 %}...{% else %}...{% endif %}`
`for`	Loops over any iterable. Inside it you get `forloop`.	`{% for author in book.authors.all %}...{% endfor %}`
`for ... empty`	Fallback block when the sequence is empty.	`{% empty %}<p>No books found.</p>`
`forloop.*`	counter, counter0, revcounter, first, last, parentloop.	`{% if not forloop.last %}, {% endif %}`
`for ... reversed`	Iterates backwards without changing the data.	`{% for book in books reversed %}`
`with`	Names a value once, useful for expensive or long expressions. Valid only inside the block.	`{% with n=page_obj.paginator.count %}{{ n }}{% endwith %}`
`cycle`	Alternates between values on each pass, handy for striped rows.	`<tr class="{% cycle "odd" "even" %}">`
`firstof`	Prints the first value that is truthy (escaped).	`{% firstof book.summary "No summary yet." %}`
`regroup`	Groups a list by an attribute. The list must already be ordered by it.	`{% regroup books by language.name as groups %}`
`ifchanged`	Renders only when a value differs from the previous pass.	`{% ifchanged book.publication_year %}<h3>...{% endifchanged %}`

regroup in full (the view orders by language, then title):

```
{% regroup books by language.name as groups %}
{% for group in groups %}
  <h2>{{ group.grouper }} ({{ group.list|length }})</h2>
  <ul>
    {% for book in group.list %}
      <li>{{ book.title }}</li>
    {% endfor %}
  </ul>
{% endfor %}
```

4. Output, URLs and forms

Tag	What it does	Catalog example
`url`	Builds a URL from a route name, so no path is hardcoded. Use the namespace. `as var` stores it instead of printing.	`{% url 'catalog:book_detail' pk=book.pk %}`
`static`	URL of a static file (needs `{% load static %}`). Respects STATIC_URL.	`{% static 'css/site.css' %}`
`now`	Current date or time in a format.	`© {% now "Y" %}`
`csrf_token`	Hidden field required in every POST form. Without it Django answers 403.	`<form method="post">{% csrf_token %}`
`querystring`	Django 5.1+. Rebuilds the current query string with changes, keeping q and other filters. Needs the request context processor.	`{% querystring page=page_obj.next_page_number %}`

Tag	What it does	Catalog example
`get_absolute_url`	Not a tag: a model method, called like an attribute. One place defines the book's URL.	`<a href="{{ book.get_absolute_url }}">`
`debug`	Dumps the context. Works only with DEBUG on and the client IP in INTERNAL_IPS, which in Docker is usually a 172.x address.	`{% debug %}`

5. Escaping and safety

Autoescape is on by default. Django converts `<`, `>`, `&`, quotes into HTML entities. That is what stops a book summary containing a script tag from running in your visitors' browsers. The tools that switch it off are the dangerous ones:

Tool	What it does	When to use it
`autoescape off`	Disables escaping for a whole block: `{% autoescape off %}...{% endautoescape %}`. `autoescape on` restores it.	Almost never. Only for content you generated yourself.
`|safe`	Marks one value as already safe HTML.	Only for trusted HTML, never for user input such as summary or bio.
`|escape` / `|force_escape`	Escapes explicitly. force_escape also works inside autoescape off.	Rarely needed, shows how it works.
`|striptags`	Removes HTML tags from a string.	Meta descriptions, plain text previews.
`|linebreaks`	Turns newlines into p and br, and escapes first.	The safe way to show a multi-line summary.
`verbatim`	Prints `{{ }}` and `{% %}` literally.	Showing template code on a page, or JS frameworks sharing the syntax.
`|json_script`	Outputs a Python value as safe JSON inside a script tag.	Passing book data to JavaScript, instead of building JSON by hand.

Classroom demo: save this summary on a book, then compare the four lines.

```
summary = "<b>Great</b> <script>alert(1)</script>"

{{ book.summary }}                shown as text, harmless
{{ book.summary|linebreaks }}     escaped, with paragraphs
{{ book.summary|safe }}           bold text AND the alert runs: XSS
{% autoescape off %}{{ book.summary }}{% endautoescape %}   same risk, whole block
```

6. Filters

A filter transforms a value: `{{ value|filter:argument }}`. They chain left to right, and the argument, when needed, follows a colon.

Filter	What it does	Catalog example
`default`	Fallback when the value is falsy (empty, 0, False, None).	`{{ book.summary|default:"None yet" }}`
`default_if_none`	Fallback only for None. Better when 0 is valid.	`{{ book.pages|default_if_none:"unknown" }}`
`length`	Items in a list or characters in a string.	`{{ books|length }}`

Filter	What it does	Catalog example
`pluralize`	Adds s depending on the number. Optional singular,plural arguments.	`{{ n }} book{{ n|pluralize }}`
`yesno`	Maps True, False and None to words.	`{{ book.available|yesno:"Available,On loan" }}`
`join`	Joins a list with a separator, calling str() on each item (so Author prints its full_name).	`{{ book.authors.all|join:", " }}`
`first / last / slice`	Pick items from a sequence.	`{{ book.genres.all|slice:":3" }}`
`title / upper / lower / capfirst`	Change case.	`{{ author.last_name|upper }}`
`truncatewords / truncatechars`	Shorten text, adding an ellipsis.	`{{ book.summary|truncatewords:25 }}`
`wordcount`	Number of words.	`{{ book.summary|wordcount }}`
`linebreaks / linebreaksbr`	Newlines to paragraphs or br tags.	`{{ author.bio|linebreaks }}`
`date`	Formats date and datetime fields. Not for publication_year, which is an integer.	`{{ author.birth_date|date:"d M Y" }}`
`timesince`	Elapsed time since a date.	`{{ book.created_at|timesince }} ago`
`floatformat`	Rounds numbers.	`{{ avg_pages|floatformat:1 }}`
`add`	Adds a number or joins strings.	`{{ page_obj.number|add:1 }}`
`slugify / urlencode`	Make text URL friendly or safe in a link.	`{{ book.title|slugify }}`
`dictsort`	Sorts a list of dicts by a key. For querysets, order in the view instead.	`{{ rows|dictsort:'year' }}`

7. The catalog templates, assembled

templates/base.html

```
{% load static %}
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>{% block title %}Book Catalog{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'css/site.css' %}">
</head>
<body>
  <nav><a href="{% url 'catalog:book_list' %}">Books</a></nav>
  {% block content %}{% endblock %}
  <footer>&copy; {% now "Y" %} Book Catalog</footer>
</body>
</html>
```

catalog/_book_card.html (the include, one card per book)

```
<article>
  <h2><a href="{{ book.get_absolute_url }}">{{ book.title }}</a></h2>
  <p>{{ book.authors.all|join:", " }} ({{ book.publication_year }})</p>
  <p>{{ book.summary|truncatewords:25|default:"No summary yet." }}</p>
  <p>{{ book.available|yesno:"Available,On loan" }}</p>
</article>
```

catalog/book_list.html

```
{% extends "base.html" %}
{% block title %}Books - {{ block.super }}{% endblock %}

{% block content %}
<h1>
  {{ page_obj.paginator.count }} book{{ page_obj.paginator.count|pluralize }}
  {% if q %}matching "{{ q }}"{% endif %}
</h1>

<form method="get">
  <input name="q" value="{{ q }}"> <button>Search</button>
</form>

{% for book in page_obj %}
  {% include "catalog/_book_card.html" with book=book only %}
{% empty %}
  <p>No books found.</p>
{% endfor %}

{% if page_obj.has_other_pages %}
  <nav>
    {% if page_obj.has_previous %}
      <a href="{% querystring page=page_obj.previous_page_number %}">Previous</a>
    {% endif %}
    Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
    {% if page_obj.has_next %}
      <a href="{% querystring page=page_obj.next_page_number %}">Next</a>
    {% endif %}
  </nav>
{% endif %}
{% endblock %}
```

catalog/book_detail.html

```
{% extends "base.html" %}
{% block title %}{{ book.title }}{% endblock %}

{% block content %}
<h1>{{ book.title }}</h1>

{% if book.cover %}
  <img src="{{ book.cover.url }}" alt="Cover of {{ book.title }}">
{% endif %}

<dl>
  <dt>Authors</dt>
  <dd>
    {% for author in book.authors.all %}
      {{ author.full_name }}{% if not forloop.last %}, {% endif %}
    {% endfor %}
  </dd>
  <dt>Genres</dt>   <dd>{{ book.genres.all|join:", "|default:"None" }}</dd>
  <dt>Language</dt> <dd>{{ book.language.name }}</dd>
  <dt>Published</dt><dd>{{ book.publication_year }}</dd>
  <dt>Pages</dt>    <dd>{{ book.pages|default_if_none:"unknown" }}</dd>
  <dt>ISBN</dt>     <dd>{{ book.isbn }}</dd>
  <dt>Status</dt>   <dd>{{ book.available|yesno:"Available,On loan" }}</dd>
  <dt>Added</dt>
  <dd>{{ book.created_at|date:"d M Y" }} ({{ book.created_at|timesince }} ago)</dd>
</dl>

{{ book.summary|linebreaks }}

<a href="{% url 'catalog:book_list' %}">Back to the list</a>
{% endblock %}
```

Performance link to the models: `book.authors.all` inside the list loop runs one query per book. In the view use `Book.objects.select_related("language").prefetch_related("authors", "genres")` and the same templates make 3 queries instead of 1 + 2 per book.

8. Your own filter and tag (optional)

Create `catalog/templatetags/__init__.py` (empty) and the module below, then restart the dev server. Python modules in `templatetags` are only discovered at startup.

```
# catalog/templatetags/catalog_extras.py
from django import template
from catalog.models import Book

register = template.Library()


@register.filter
def reading_time(pages):
    """Rough reading time in hours, assuming 40 pages per hour."""
    if not pages:
        return ""
    return f"{pages / 40:.1f} h"


@register.simple_tag
def available_count():
    return Book.objects.filter(available=True).count()
```

```
{% load catalog_extras %}
{{ book.pages|reading_time }}
{% available_count as n %}  {{ n }} books on the shelf
```

9. Common mistakes

Symptom	Cause and fix
A value prints nothing	Typo or wrong path. Debug with `{{ book|pprint }}` or `{% debug %}`.
Prints `catalog.Author.None`	You printed a related manager. Add `.all`.
Child page shows no content	Content sits outside a block, or `extends` is not the first tag.
static or humanize 'is not a registered tag library'	`{% load %}` is per template, not inherited.
`NoReverseMatch`	Wrong route name, missing namespace (`catalog:`), or missing arguments.
Method with arguments cannot be called	Move the logic to the view, a model property, or a custom tag or filter.
Page is slow with many books	N+1 queries from related fields in a loop. Use select_related and prefetch_related.
Script from user text runs	`|safe` or `autoescape off` on user content. Remove it.
Include sees too much or too little	Add `with` and `only` so the include's inputs are visible.

10. Teaching flow for 75 minutes

Min	Block	Goal
10	base.html with extends, block, static, url, now	Inheritance and one source of layout.
20	List page: for, empty, forloop, if/elif, with	Logic in templates, and where to stop.

Min	Block	Goal
15	include, querystring, pagination	Reusable pieces and filters that survive paging.
15	Detail page and filters: join, yesno, date, default_if_none	Presenting related models.
15	Autoescape demo, silent failures, N+1 with the debug toolbar or logging	Safety and debugging habits.